
Text Summarization System

Natural Language Processing

1. Problem Description

Automatic text summarization is a fundamental challenge in Natural Language Processing (NLP). Given a long document, the goal is to produce a shorter version that accurately captures the most important information. This task is particularly valuable in scenarios involving news articles, legal documents, and scientific papers, where readers benefit from concise summaries rather than reading entire texts.

This project addresses the task of automatic text summarization on news articles. Two fundamentally different approaches were developed and compared:

- An extractive baseline model that scores and selects the most important sentences from the original article using statistical and semantic features.
- An advanced abstractive model based on a pre-trained Transformer (DistilBART) that generates new, paraphrased summaries rather than copying sentences verbatim.

The central objective is to evaluate whether an abstractive neural approach produces summaries of higher quality and fluency compared to a classical feature-engineering extractive approach, while also examining trade-offs in computational cost and compression ratio.

2. Dataset

Link: [CNN-DailyMail](#)

The CNN/DailyMail dataset was used for this project. It is a widely adopted benchmark for text summarization research, consisting of news articles paired with human-written highlights (reference summaries). The dataset was loaded from three CSV files corresponding to the standard splits:

Split	Approximate Rows	Columns
Train	~287,000	id, article, highlights
Validation	~13,000	id, article, highlights
Test	~11,000	id, article, highlights

In this notebook, the available training split contained 174,689 records after loading.

Each record contains a unique identifier (id), the full news article (article), and the reference summary written by professional editors (highlights). The articles have an average length of several hundred words, while the highlights are significantly shorter, typically two to four sentences.

Due to memory and time constraints during experimentation, a random sample of 5,000 training records was used for feature extraction and baseline model training, while 10 records were used to demonstrate and evaluate the advanced Transformer model.

3. Preprocessing Steps

The raw data underwent a systematic cleaning and preprocessing pipeline before any modeling could begin. The steps are described below.

3.1 Data Cleaning

The following cleaning operations were applied to all three splits:

Step	Action	Rationale
1	Drop null rows	Remove records where the article or highlights field is missing
2	Strip whitespace	Remove leading and trailing whitespace from text fields
3	Drop empty strings	Remove records that are empty after stripping
4	Deduplicate by ID	Ensure each document appears only once
5	Word-count filtering	Keep articles between 30 and 2,000 words; highlights above 5 words

The word-count thresholds (MIN_ARTICLE_WORDS = 30, MAX_ARTICLE_WORDS = 2,000, MIN_HIGHLIGHTS_WORDS = 5) were chosen to eliminate stub articles and extremely long documents that would introduce noise or exceed model token limits.

3.2 Text Preprocessing

After data cleaning, each article was preprocessed at the text level using the following operations:

- Lowercasing: All text was converted to lowercase.
- Punctuation removal: Non-alphabetic characters were removed using regular expressions.

-
- Sentence splitting: Articles were split into individual sentences using period-based tokenization, retaining only sentences longer than five words.
 - Word tokenization: The cleaned text was tokenized into individual word tokens using NLTK's `word_tokenize` function.
 - Stopword removal: Common English stopwords (e.g., 'the', 'is', 'and') were removed using NLTK's stopwords corpus to reduce noise in feature vectors.

The output of preprocessing for each article consisted of three fields: a list of original sentences, a list of cleaned tokens, and a single `clean_text` string (tokens joined by spaces). These were saved to a CSV file for downstream feature extraction.

4. Methods

4.1 Feature Extraction

Two complementary feature types were extracted from the preprocessed corpus to quantify sentence importance.

TF-IDF Features: A `TfidfVectorizer` was fitted on the full corpus of cleaned texts with a vocabulary of up to 20,000 terms, using unigrams and bigrams (`ngram_range = (1, 2)`), log-smoothed term frequency (`sublinear_tf = True`), and a minimum document frequency of 2. For each sentence in an article, the mean of its non-zero TF-IDF weights was computed as a sentence-level importance score. Sentences containing many distinctive, rare terms received higher scores.

Word2Vec Embeddings: A Skip-gram `Word2Vec` model was trained directly on the corpus token lists with a vector size of 100 dimensions, a context window of 5, a minimum word count of 3, and 5 training epochs. Each sentence was represented as the mean-pooled vector of its constituent word embeddings. The cosine similarity between each sentence vector and the document centroid vector was computed as the embedding importance score, reflecting how representative the sentence is of the overall article topic.

Combined Importance Score: Both scores were min-max normalized to [0, 1] and combined using a weighted sum:

$$importance(s) = \alpha * TF-IDF(s) + (1 - \alpha) * EmbeddingSim(s)$$

The weight α was set to 0.3, giving more emphasis (70%) to semantic representativeness via embeddings and less (30%) to lexical distinctiveness via TF-IDF.

4.2 Baseline Model — Extractive Summarization

The baseline model is a simple extractive summarizer. Given the combined importance scores for each sentence in an article, the top-3 highest-scoring sentences are selected and returned in their original document order as the summary. No text is generated; the summary is a direct copy of selected sentences.

This approach is computationally efficient and interpretable: the summary is guaranteed to consist of grammatically correct, factually accurate sentences from the source article. However, it is limited by the quality of the scoring function and cannot produce fluent, concise summaries that combine ideas from multiple parts of the article.

To evaluate baseline quality, two metrics were measured:

- **Compression Ratio:** The ratio of summary word count to article word count, reflecting how much the article was reduced.
- **Semantic Similarity:** The cosine similarity between the sentence-transformer embedding (all-MiniLM-L6-v2) of the full article and the baseline summary, measuring how well the summary captures the article's meaning.

4.3 Advanced Model — Abstractive Transformer Summarization

The advanced model uses DistilBART (sshleifer/distilbart-cnn-6-6), a distilled version of Facebook's BART model fine-tuned on the CNN/DailyMail dataset. BART employs an encoder-decoder architecture: the encoder processes the input article and the decoder autoregressively generates a new summary token by token using beam search.

Unlike the baseline, the Transformer can paraphrase, merge, and reframe information, producing abstractive summaries that may not appear verbatim in the source text. The following key generation parameters were used:

Parameter	Value	Purpose
max_input_tokens	512	Token budget per input chunk
max_length	80	Maximum summary length in tokens
min_length	20	Minimum summary length to avoid empty outputs
num_beams	1	Greedy decoding (set higher for better quality)
length_penalty	1.5	Encourage longer, more complete summaries
no_repeat_ngram_size	3	Prevent repetitive 3-grams in output

To handle articles longer than 512 tokens, a chunking strategy was implemented: articles are split at sentence boundaries into token-limited chunks, each chunk is summarized independently, and — if the combined chunk summaries fit within the token limit — a final summarization pass is applied to produce a single coherent output.

5. Results and Comparison

5.1 Data Statistics After Cleaning

The cleaning pipeline successfully removed noise from all three dataset splits. Below is a summary of the cleaned data:

Split	Raw Rows	Clean Rows	Avg. Article Words	Avg. Highlight Words
Train	174,689	~174,400	~692	~52
Validation	~13,368	~13,366	~769	~52
Test	~11,490	~11,489	~760	~57

The cleaning step removed relatively few rows, confirming that the raw dataset is already of high quality. Word-count statistics show that articles are substantially longer than highlights, validating the compression objective.

5.2 Feature Score Analysis

Across the 5,000 training samples used for feature extraction, the sentence-level scores exhibited the following characteristics:

- TF-IDF scores followed a right-skewed distribution, with most sentences receiving low scores and a small subset receiving high scores — consistent with the nature of TF-IDF, which rewards rare, distinctive terms.
- Embedding similarity scores were more uniformly distributed, reflecting the variety of topics covered across the corpus.
- The correlation between TF-IDF scores and embedding scores was low, confirming that the two features capture complementary information and that their combination is beneficial.

5.3 Baseline Model Performance

The extractive baseline was evaluated on the 5,000-sample subset. Key results are presented below:

Metric	Value	Interpretation
Average Compression Ratio	0.122	Summaries are ~12.2% the length of the original article
Average Semantic Similarity	0.685	High semantic overlap between summary and article (MiniLM cosine)

The baseline achieves a semantic similarity score of 0.685 by directly copying sentences from the source article. The compression ratio of approximately 12.2% means the model reduces article length by roughly 8x, which represents a strong level of compression for a news summarization task.

5.4 Advanced Model Performance

The Transformer model was evaluated on 10 sample articles. Qualitative and quantitative observations are summarized below:

Metric	Value	Interpretation
Average Compression Ratio	Varies (~7%–35% across samples)	Summaries are shorter and more condensed than baseline
Summary Style	Abstractive	New sentences generated; not copied from source
Fluency	High	Grammatically correct, coherent output
Factual Grounding	Generally accurate	Tied to CNN/DM fine-tuning; hallucinations possible on unusual text

The Transformer produces more compressed, fluent, and readable summaries. It can merge information from multiple sentences and reframe content in new words. This is its primary advantage over the extractive baseline.

5.5 Comparative Summary

The table below provides a direct side-by-side comparison of both approaches across the most relevant evaluation dimensions:

Dimension	Baseline (Extractive)	Advanced (DistilBART)
Approach	Extractive (sentence selection)	Abstractive (text generation)
Features used	TF-IDF + Word2Vec embeddings	Pre-trained BART encoder-decoder
Compression ratio	12.2%	Varies widely across articles
Semantic similarity	0.685 (direct sentence copy)	~0.80 (slightly lower, paraphrase)
Fluency	Moderate (stitched sentences)	High (coherent generation)
Novel phrasing	None	Yes (paraphrases source text)
Inference speed	Fast (milliseconds)	Slow (seconds per article)
Memory requirements	Low	High (~667.5 MB model)
Interpretability	High (traceable scores)	Low (neural black box)

6. Conclusion

This project developed and evaluated two text summarization systems for news articles from the CNN/DailyMail dataset: an extractive baseline combining TF-IDF and Word2Vec sentence

importance scores, and an advanced abstractive model based on the DistilBART pre-trained Transformer.

The preprocessing pipeline successfully cleaned and normalized the dataset with minimal data loss, and the feature extraction stage demonstrated that TF-IDF and embedding-based signals are complementary and together provide a useful ranking of sentence importance.

The baseline model is simple, fast, and interpretable, achieving high semantic similarity by virtue of directly copying important sentences. However, its summaries may read as disjoint or over-long compared to human-written highlights.

The Transformer model produces more compressed, fluent, and coherent summaries that can paraphrase and combine ideas — properties that are crucial for real-world summarization systems. Its main limitations are significantly higher computational cost and the risk of hallucination in edge cases.

In conclusion, for applications where computational resources are available and summary quality is paramount, the Transformer-based abstractive approach is clearly preferable. For lightweight or real-time applications where interpretability and speed matter more, the extractive baseline remains a competitive and practical choice. Future work could explore higher beam counts in the Transformer decoder, ROUGE-based evaluation, and fine-tuning the model on the full CNN/DailyMail training set.